Họ và tên: Đoàn Anh Quân

MSSV: 23110111

BÀI THỰC HÀNH DATA MINING TUẦN 5

Cài đặt và thực thi code thuật toán Vertical Apriori:

- Import các thư viện và load dữ liệu:

```
import numpy as np
import pandas as pd

df = pd.read_csv(r"D:\data_mining\tuan5\data.csv")
print(df)
```

✓ 0.0s

```
    TID                                Itemset
0     1  Wine, Chips, Bread, Butter, Milk, Apple
1     2                Wine, Bread, Butter, Milk
2     3                       Bread, Butter, Milk
3     4                              Chips, Apple
4     5  Wine, Chips, Bread, Butter, Milk, Apple
5     6                        Wine, Chips, Milk
6     7        Wine, Chips, Bread, Butter, Apple
7     8                        Wine, Chips, Milk
8     9                       Wine, Bread, Apple
9    10                Wine, Bread, Butter, Milk
10   11              Chips, Bread, Butter, Apple
11   12                Wine, Butter, Milk, Apple
12   13         Wine, Chips, Bread, Butter, Milk
13   14                 Wine, Bread, Milk, Apple
14   15         Wine, Bread, Butter, Milk, Apple
15   16  Wine, Chips, Bread, Butter, Milk, Apple
16   17        Chips, Bread, Butter, Milk, Apple
17   18               Chips, Butter, Milk, Apple
18   19  Wine, Chips, Bread, Butter, Milk, Apple
19   20         Wine, Bread, Butter, Milk, Apple
20   21          Wine, Chips, Bread, Milk, Apple
21   22                                    Chips
```

- Tạo dataset_dict gồm tên 1 loại vật phẩm và các lần xuất hiện của nó:

```
df['Itemset'] = df['Itemset'].str.split(r',\s*')
df_exploded = df.explode('Itemset')
dataset_dict = df_exploded.groupby('Itemset')['TID'].apply(set).to_dict()

print(dataset_dict)
```

✓ 0.0s

{'Apple': {1, 4, 5, 7, 9, 11, 12, 14, 15, 16, 17, 18, 19, 20, 21},

'Bread': {1, 2, 3, 5, 7, 9, 10, 11, 13, 14, 15, 16, 17, 19, 20, 21},

'Butter': {1, 2, 3, 5, 7, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20},

'Chips': {1, 4, 5, 6, 7, 8, 11, 13, 16, 17, 18, 19, 21, 22},

'Milk': {1, 2, 3, 5, 6, 8, 10, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21},

'Wine': {1, 2, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 19, 20, 21}}

- Thuật toán vertical_aptiori. Code sẽ trả về list toàn bộ cách kết hợp các item và số lần xuất hiện của cách kết hợp đó lớn hơn minsup:

```python
from itertools import combinations

def vertical_apriori(tid_lists_dict, min_combination):

    k = 1
    F_k = {}

    for item, tids_set in tid_lists_dict.items():
        if len(tids_set) >= min_combination:
            F_k[(item,)] = tids_set

    all_frequent_itemsets = F_k.copy()

    while F_k:
        C_k_plus_1 = {}
        F_k_items = list(F_k.keys())
        n = len(F_k_items)

        for i in range(n):
            for j in range(i + 1, n):
                itemset1 = F_k_items[i]
                itemset2 = F_k_items[j]

                if itemset1[:-1] == itemset2[:-1]:
                    candidate = tuple(sorted(set(itemset1) | set(itemset2)))

                    is_valid = True
                    for subset in combinations(candidate, k):
                        if subset not in F_k:
                            is_valid = False
                            break

                    if is_valid:
                        tid_list1 = F_k[itemset1]
                        tid_list2 = F_k[itemset2]
```

✓ 0.0s

```python
                        intersected_tids = tid_list1.intersection(tid_list2)
                        C_k_plus_1[candidate] = intersected_tids

        F_k_plus_1 = {}

        for candidate, tids in C_k_plus_1.items():
            support = len(tids)
            if support >= min_combination:
                F_k_plus_1[candidate] = tids

        all_frequent_itemsets.update(F_k_plus_1)
        F_k = F_k_plus_1
        k += 1

    return all_frequent_itemsets
```

✓ 0.0s

- Chạy thuật toán Vertical apiori với min_combination = 2:

```python
minsup = 4
frequent_itemsets = vertical_apriori(dataset_dict, minsup)

for itemset, tids in frequent_itemsets.items():
    print(f"{itemset}: {len(tids)}")
```

✓ 0.0s

```
('Apple',): 15
('Bread',): 16
('Butter',): 15
('Chips',): 14
('Milk',): 17
('Wine',): 16
('Apple', 'Bread'): 12
('Apple', 'Butter'): 11
('Apple', 'Chips'): 10
('Apple', 'Milk'): 11
('Apple', 'Wine'): 11
('Bread', 'Butter'): 13
('Bread', 'Chips'): 9
('Bread', 'Milk'): 13
('Bread', 'Wine'): 13
('Butter', 'Chips'): 9
('Butter', 'Milk'): 13
('Butter', 'Wine'): 11
('Chips', 'Milk'): 10
('Chips', 'Wine'): 9
('Milk', 'Wine'): 14
('Apple', 'Bread', 'Butter'): 9
('Apple', 'Bread', 'Chips'): 8
('Apple', 'Bread', 'Milk'): 9
('Apple', 'Bread', 'Wine'): 10
...
('Apple', 'Bread', 'Chips', 'Milk', 'Wine'): 5
('Apple', 'Butter', 'Chips', 'Milk', 'Wine'): 4
('Bread', 'Butter', 'Chips', 'Milk', 'Wine'): 5
('Apple', 'Bread', 'Butter', 'Chips', 'Milk', 'Wine'): 4
```

- Tạo 1 dataset result gồm itemset là các item, count là số lần xuất hiện của itemset đó, item_count là số lượng item trong itemset đó, tid list là danh sách các vị trí xuất hiện của itemset đó, support là độ support của itemset đó. Lọc các dòng để số thỏa min_support:

```
min_support = 0.6
results = []

for itemset, tids in frequent_itemsets.items():
    new_colums = {
        'Itemset': ", ".join(itemset),
        'Count': len(tids),
        'Item_count': len(itemset),
        'TID list': tids,
        'Support': (len(tids) / df.shape[0])
    }
    results.append(new_colums)

new_df = pd.DataFrame(results)

new_df = new_df[new_df['Support'] >= min_support]
new_df
✓ 0.0s
```

	Itemset	Count	Item_count	TID list	Support
0	Apple	15	1	{1, 4, 5, 7, 9, 11, 12, 14, 15, 16, 17, 18, 19...	0.681818
1	Bread	16	1	{1, 2, 3, 5, 7, 9, 10, 11, 13, 14, 15, 16, 17,...	0.727273
2	Butter	15	1	{1, 2, 3, 5, 7, 10, 11, 12, 13, 15, 16, 17, 18...	0.681818
3	Chips	14	1	{1, 4, 5, 6, 7, 8, 11, 13, 16, 17, 18, 19, 21,...	0.636364
4	Milk	17	1	{1, 2, 3, 5, 6, 8, 10, 12, 13, 14, 15, 16, 17,...	0.772727
5	Wine	16	1	{1, 2, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, ...	0.727273
20	Milk, Wine	14	2	{1, 2, 5, 6, 8, 10, 12, 13, 14, 15, 16, 19, 20...	0.636364

- Tạo support_dict với các item riêng lẻ để tìm độ support tương ứng dễ dàng hơn:

```
df_1_item = new_df[new_df['Item_count'] == 1]
support_dict = dict(zip(df_1_item['Itemset'], df_1_item['Support']))
support_dict
✓ 0.0s
```

```
{'Apple': 0.6818181818181818,
 'Bread': 0.7272727272727273,
 'Butter': 0.6818181818181818,
 'Chips': 0.6363636363636364,
 'Milk': 0.7727272727272727,
 'Wine': 0.7272727272727273}
```

- Lọc các dòng new_df với min_combination:

```
min_combination = 2
new_df = new_df[new_df['Item_count'] >= min_combination]
new_df
```

✓ 0.0s

	Itemset	Count	Item_count	TID list	Support
20	Milk, Wine	14	2	{1, 2, 5, 6, 8, 10, 12, 13, 14, 15, 16, 19, 20...	0.636364

- Tạo 1 dataset rules tính toán các luật trong đó dựa vào new_df:

```
min_confidence = 0
rules = []
for i in range(new_df.shape[0]):
    if new_df.iloc[i, 2] == 2:
        string = new_df.iloc[i, 0]
        item_A = [item.strip() for item in string.split(',')][0]
        item_B = [item.strip() for item in string.split(',')][1]

        conf_A_B = new_df.iloc[i, 4] / support_dict[item_A]
        if conf_A_B >= min_confidence:
            rules.append([item_A, item_B, float(new_df.iloc[i, 4]), float(conf_A_B)])

        conf_B_A = new_df.iloc[i, 4] / support_dict[item_B]
        if conf_B_A >= min_confidence:
            rules.append([item_B, item_A, float(new_df.iloc[i, 4]), float(conf_B_A)])

rules = pd.DataFrame(rules, columns=['Antecedents', 'Consequents', 'Support', 'Confidence'])
print(rules)
```

✓ 0.0s

	Antecedents	Consequents	Support	Confidence
0	Milk	Wine	0.636364	0.823529
1	Wine	Milk	0.636364	0.875000

So sánh code thuật toán Vertical Apriori có sẵn và tự code:

- Code mẫu đáp án:

	Item	Support
0	Milk & Wine	0.636364

```
rule_indeces
✓ 0.0s
{'Milk & Wine': [0, 1, 4, 5, 7, 9, 11, 12, 13, 14, 15, 18, 19, 20]}
```

```
rule_supports
✓ 0.0s
{'Milk & Wine': 0.6363636363636364}
```

- Tự code đáp án:

	Itemset	Count	Item_count	TID list	Support
20	Milk, Wine	14	2	{1, 2, 5, 6, 8, 10, 12, 13, 14, 15, 16, 19, 20...	0.636364

	Antecedents	Consequents	Support	Confidence
0	Milk	Wine	0.636364	0.823529
1	Wine	Milk	0.636364	0.875000

⇨ Giống nhau đáp án nhưng vì đánh số khác nhau nên TID list bị lệch 1 đơn vị.